CER: The LLM COMPASS: Aligning Values and Code through Rhetorical AI and Continuous Intent Verification

Tim Menzies (PI) and David Rieder (Co-PI) and Sarah Heckman (SP) NC State University

1 Overview

Modern AI compresses software lifecycles. Hyper-productive developers can now leap from ideation to code, refining requirements implicitly rather than systematically [1, 2, 3]. Consequently, developers can face an overwhelming avalanche of rapidly changing implementation details. Such rapid updates risk **design drift**: a divergence from core values where speed overrides socio-technical goals, favoring narrow technical solutions [3].

How can developers retain the "big picture" [4] while managing an avalanche of rapidly changing implementation details? Traditional Requirements Engineering (RE) cannot keep pace. Structured as an up-front activity, it provides little real-time guidance. Furthermore, static warnings are easily ignored by busy coders; to be effective, tools must be *suasive* [5]. Yet, RE education remains neglected:

"RE education is at best an afterthought in many university software engineering curricula" [6].

To support RE at the speed of modern AI, Figure 1 proposes **COMPASS**, a tool that uses interface design and persuasive triggers to help students understand and act on key requirements principles. COMPASS extends standard RAG [7] by integrating **Suasive Interface Theory** [5]. Given assessment documents (Table 1), COMPASS uses RAG to review artifacts against background material (legislation, safety rules). However, unlike passive linters, it uses LLMs to *persuade* developers to align with these values through timely prompts and interpretive cues that frame design decisions and highlight risks.

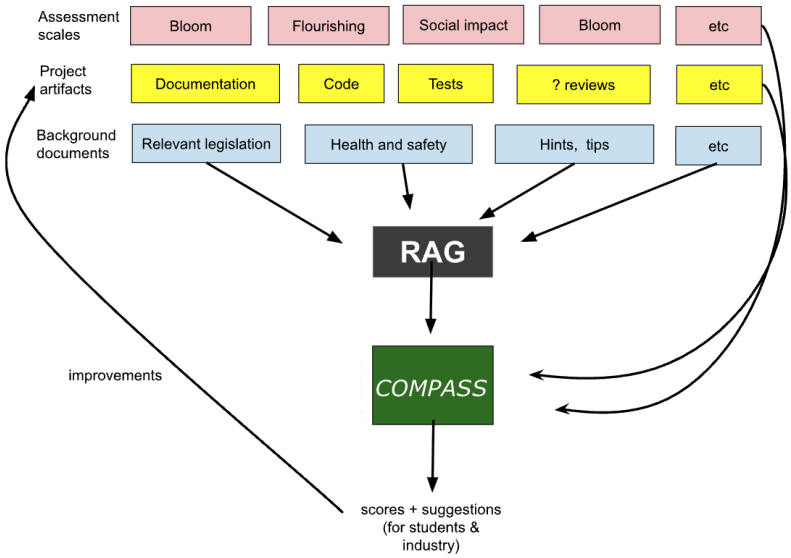

Figure 1: COMPASS v1.0: A Suasive RAG Architecture.

COMPASS addresses a gap by focusing on graduate-level RE rather than introductory coding. Most education research targets "hands-on-keyboard" literacy (K-12/CS1)[8, 9]. Why is RE neglected? Perhaps because it traditionally required years of experience to recognize missing requirements. Now, empowered by large language models (LLMs) and rhetorical theory, COMPASS can:

1. Import required background knowledge;
2. Score projects using Context-Aware Dimensions;
3. **Construct suasive arguments** (via LLM agents) to motivate improvements;
4. Automates tasks 1–3, giving every student / software project a personal **Rhetorical Coach**.

For examples of scoring scales, see Table 1. We stress that COMPASS is non-prescriptive: it does not demand that teams evaluate themselves with respect to some pre-determined criteria. Rather, through iterative interaction with rhetorically designed interventions, COMPASS helps teams progressively refine their own ethical or legal criteria (e.g., Table 2) rather than imposing external values.

3 Preliminary Results

We say that **Continuous Intent Verification (CIV)** is the ongoing assessment of a project's alignment with its goals. Before investing in the rhetorical interfaces required to deploy CIV in academia and industry (see next section), we first had to establish a fundamental baseline: *Can current LLMs accurately detect strategic misalignments in graduate-level software projects?*

To answer this, we ran an exploratory study comparing student projects from Fall 2024 (Control) and Fall 2025 (Experimental). That study found that a prototype version of COMPASS (lacking the full suasive interface but possessing the core scoring engine) successfully shifted a cohort towards higher innovation.

3.1 Experimental Context and Methodology

To assess the pedagogical and analytical capabilities of COMPASS, we conducted an exploratory empirical study that compared student projects from Fall 2024 and Fall 2025. The methods of this project were absent from Fall 2024 and were only applied to the Fall 2025 class.

Longitudinal data from the preceding decade (2015–2025) of this course revealed a persistent trend toward conventional project themes (e.g., generic "exercise trackers" or "menu assistants"). To mitigate this *innovation blindspot*, we introduced an LLM-supported ideation phase.

Task Protocol: Student teams were provided with ten baseline use cases for a food delivery system. Utilizing OpenAI Python packages, teams explored a domain-specific corpus comprising approximately

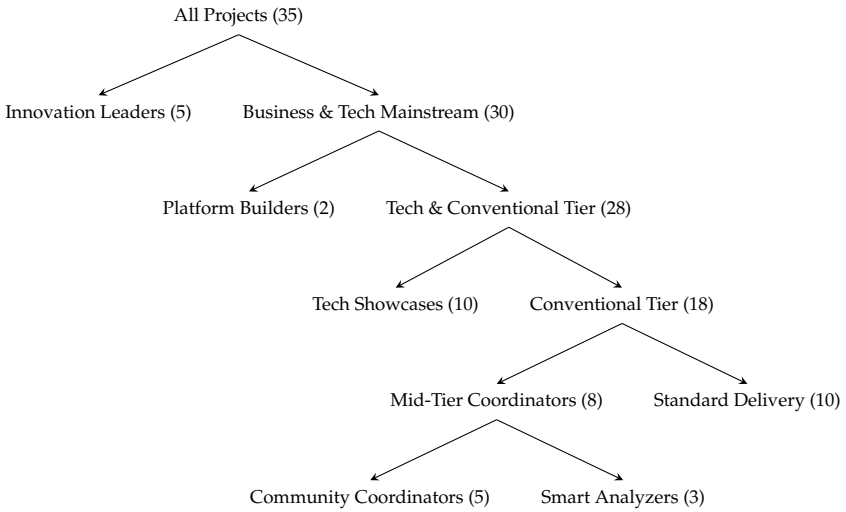

Figure 2: Hierarchy of student project clusters (2025 data) report by COMPASS's LLMs (data from Table 4 based on 35 student projects).

Cluster	Definition & General Impact
Innovation Leaders	High social impact with strong wellbeing benefits across multiple life dimensions. Balance technical execution with genuine societal value through health, social connections, and community benefit.
Platform Builders	Infrastructure-focused projects enabling ecosystems through APIs and extensibility. Demonstrate platform economics and architectural openness for third-party value creation.
Tech Showcases	Sophisticated AI/LLM applications with high cognitive complexity. Excel in novel algorithms but focus on individual benefit without broader social impact.
Community Coordinators	Emphasize local social connection within neighborhoods or campus. Facilitate group interactions, shared resources, and collective benefit at human scale.
Smart Analyzers	Mid-tier intelligence with recommendation engines or optimization algorithms. Apply analytical thinking for individual decision-making without systemic reach.
Standard Delivery	Conventional food delivery or marketplace implementations. Demonstrate technical competence but lack differentiation in impact or business model.

Table 3: Student Project Cluster Definitions and General Impact

Scale	**Business Oriented** Ref: [10] 10,000+ citations	**Social Impact** Ref: [11] 7,500 citations	**Human Flourishing** Ref: [12] 2,500+ citations	**Bloom's** taxonomy Ref1: [13] 85,000+ citations Ref2: [14] 45,000+ citations
level=1 (lowest)	One customer type, one value proposition.	**Individual.** One person at a time, no network effects.	**Narrow impact.** Convenience only, or effects just one domains.	**Remember/Understand.** Recalls or explains existing concepts.
level=2	**Multi-Sided.** Multiple customer segments, different value props.	**Community.** Neighborhood, campus, or small group benefits.	**Limited wellbeing.** Affects domains moderately.	**Apply (Basic).** Uses known techniques in standard ways.
level=3	**Platform (Early).** Enables limited 3rd-party participation, has APIs.	**Institutional.** Organization-wide or city-level systems.	**Moderate wellbeing.** Affects domains.	**Apply (Advanced).** Well-executed application, multiple integrations.
level=4	**Platform (Mature).** Robust ecosystem, 3rd parties build businesses.	**Societal.** Regional, industry-wide impact, potential policy influence.	**Substantial wellbeing.** Affects domains meaningfully.	**Analyze/Evaluate.** Breaks down problems, systems for evaluation.
level=5 (highest)	**Infrastructure.** Industry foundational layer, platforms built on it.	**Systemic.** International transformation, new infrastructure.	**Holistic flourishing.** Transforms multiple life dimensions.	**Create.** Produces genuinely novel solutions or combinations.
Might be used by...	Entrepreneurs, product managers, platform strategists.	Social entrepreneurs, policy makers, community organizers.	Ethicists, HCI researchers, human-centered designers.	Educators, curriculum designers, technical hiring managers.

Table 1: Domain General Evaluation Dimensions for Software Projects. Using LLMs, it is possible to summarize and score a wide range of software project artifacts on these scales. Also, LLMs can propose minimal changes that most improve a project (where "improve" means "drive it higher up these scales").

Scale	**Medical Records** (HIMSS EMRAM) Ref: [15]	**Cybersecurity** (OWASP SAMM) Ref: [16] Industry Standard	**Green Software** (GSMM) Ref: [17] Linux Foundation	**Reliability** (Well-Architected) Ref: [18] Cloud Standard
level=1 (lowest)	**Ancillaries Only.** Lab, Pharmacy, and Radiology systems not integrated.	**Ad-hoc.** Reactive security, developers perform basic checks only.	**Unaware.** No tracking of emissions; efficiency is accidental.	**Reactive.** Failures are handled manually after they occur.
level=2	**CDR Implemented.** Clinical Data Repository exists; might have conflict checking.	**Structured.** Repeatable security activities; basic checklists used.	**Aspiring.** Ad-hoc measurement of some emissions (e.g., server count).	**Defined.** Basic redundancy (backups) and recovery procedures exist.
level=3	**Documentation.** Nursing/clinical vitals and flow sheets are electronic.	**Consistent.** Comprehensive security auditing and architecture review.	**Acting.** Tracking Scope 1, 2, & 3 emissions systematically.	**Managed.** Automated health monitoring and alerting systems.
level=4	**CPOE & CDSS.** Computerized orders with clinical decision support active.	**Measured.** Metrics-driven security improvements; defects tracked.	**Optimizing.** Software logic supports demand shifting/shaping.	**Predictive.** Automated failover; recovery tested via game days.
level=5 (highest)	**Closed Loop.** Full EMR; analytics for population health & governance.	**Optimized.** Security fully automated in CI/CD; feedback loops active.	**Carbon-Aware.** Real-time adaptation to grid carbon intensity.	**Chaos-Ready.** Fault injection; system self-heals (Anti-fragile).
Might be used by...	Health informatics, hospital administrators, compliance officers.	CISOs, DevSecOps engineers, penetration testers.	Sustainability officers, cloud architects, Green IT leads.	Site Reliability Engineers (SREs), DevOps, Platform teams.

Table 2: Domain Specific Evaluation Dimensions for Software Projects. While Table 1 was very board, this table shows domain specific maturity models (Healthcare, Security, Sustainability, Reliability).

	%						
	2024 (n=25)	2025 (n=35)	Bloom's	Flourish	Impact	Business	Profile
Innovation Leaders	4	14.3	3.8	4.0	4.0	2.0	High social impact with strong wellbeing benefits
Platform Builders	0	5.7	4.5	2.0	2.5	4.0	Ecosystem thinking, infrastructure focus
Tech Showcases	28	28.6	5.0	2.4	2.4	2.3	High AI/LLM innovation, individual benefit focus
Community Coordinators	4	14.3	3.6	2.4	2.6	2.0	Local social connection emphasis
Smart Analyzers	32	8.6	4.0	1.7	2.0	2.0	Mid-tier intelligence with tracking features
Standard Delivery	32	28.6	3.0	1.8	1.9	1.9	Conventional food delivery implementations

Table 4: Clusters found by in student projects, using the criteria of Table 1 (comparing 2024 (n=25) and 2025 (n=35) cohorts).

10,000 pages of legal, medical, and regulatory texts. Students employed a Retrieval-Augmented Generation (RAG) workflow [7] to query this corpus, identifying underserved stakeholders and asking "what requirements are missing". Students were also instructed to ask the LLMs, what changes could be made to the requirements to produce a more distinctive product.

Assessment Protocol: The COMPASS prototype utilized the Ollama environment to enable local, privacy-preserving execution on standard hardware. The automated assessment pipeline consisted of:

1. **Dimension Ingestion:** Loading the criteria definitions from Table 1 (e.g. [13, 11]).
2. **Artifact Review:** Ingesting student project documentation into the "project" directory.
3. **Scoring and Feedback:** Prompting the LLM to score artifacts against the dimensions and generate minimal viable interventions for score improvement.

To empirically categorize the project outcomes, we applied Greedy Agglomerative Clustering to the COMPASS-generated scores. This analysis yielded the hierarchical taxonomy illustrated in Figure 2. The semantic definitions of these clusters are detailed in Table 3, with quantitative distributions provided in Table 4 and Figure 3. The most notable feature of these results was that the LLMs had changed the kinds of projects generated by students, In 2024, two-thirds of the cohort's projects were in the bottom two-thirds of the scales. But in 2025, the situation was reversed with two-thirds of the projects coming from the upper scales (and some, in 2025, even creating "innovation leader" projects).

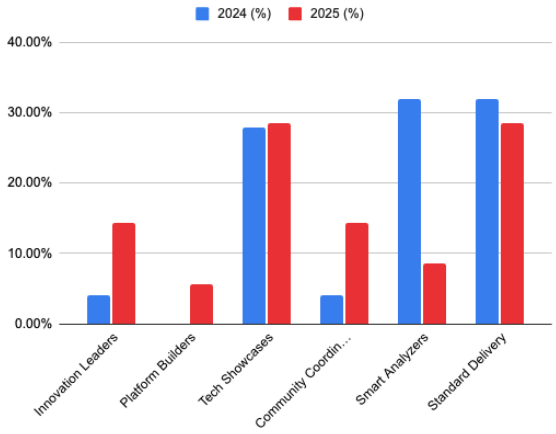

Figure 3: Student project types in 2024 (no COMPASS) and 2025 (with COMPASS).

For us, Figure 3 was a pivotal result. After years of uninspired, repetitive student projects (e.g., "calorie tracker", "exercise guide"), the 2025 projects demonstrated breakthrough levels of adaptability and creative depth. Contrary to historical baselines, COMPASS's intervention lead to significant innovation. While 10 projects were still "Standard Delivery" cluster, 25 projects demonstrated sophisticated architectural reasoning; e.g. the "Innovation Leaders" cluster (n = 5) developed meta-tools for software engineering. For example:

- *ReqTrace* (Figure 4.a) utilizes natural language processing to automatically generate knowledge graphs for conflict detection in requirements specifications.
- *Clarity-AI* (Figure 4.b) combines LLMs with NLP to infer edge cases in the requirements. These test cases are then converted to candidate test cases which the user can review, refine, reject, etc.

A decade ago, a tool like ReqTrace and Clarity-AI would be nearly science fiction. Now, thanks to LLMs and COMPASS's ideation phase, students can be inspired to build a deliver such a tool in just a few weeks.

Better yet, it was observed that COMPASS's utility extends beyond static assessment to *active enhancement*. To illustrate, we analyzed "App0," a project initially classified in the lowest-performing tier (Total Score: 7). COMPASS automatically proposed specific architectural transformations: (1) integrating philanthropic APIs; (2) implementing group booking features to enhance social impact; and (3) exposing public APIs for local business integration. The system projected that these changes would elevate App0 to the "Community Coordinators" cluster (Projected Score: 12), demonstrating the tool's capacity for specific, actionable guidance.

Furthermore, COMPASS facilitated *portfolio-level* analytics. Aggregated analysis revealed systemic deficiencies, including low "Flourishing" scores in 71% of projects and a lack of platform extensibility in 83% of submissions. Based on these diagnostics, COMPASS recommended curriculum-level interventions, such as mandating a "Flourishing Impact Statement." Simulations suggest this policy would increase the cohort's mean Flourishing score by 47%, validating COMPASS as a strategic instrument for integrating value-driven requirements and supporting educational program optimization.

Figure 4.a: ReqTrace: finds conflicting requirements.

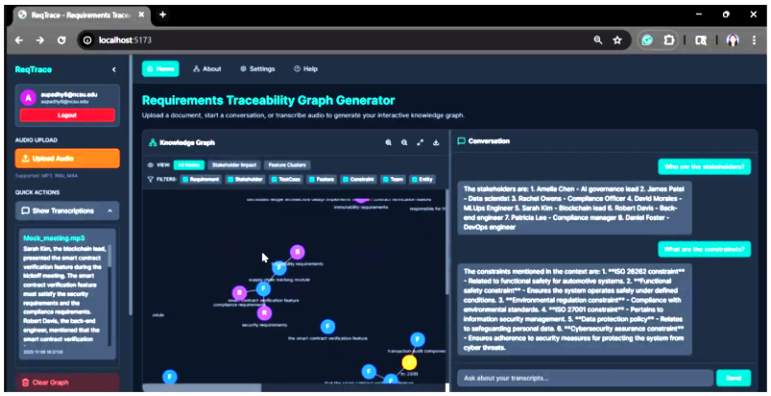

Figure 4.b: Clarity-ai: generates test cases from requirements

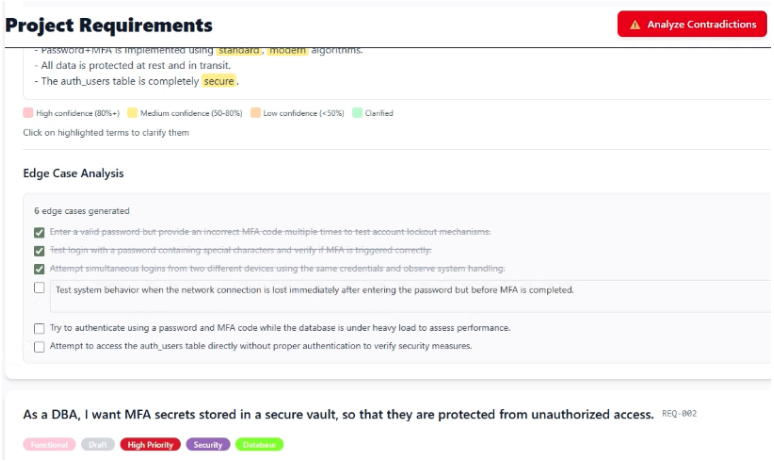

Figure 4: Two innovation leader projects from 2025.

4 Proposed Extensions: Operationalizing CIV

The prototype version of COMPASS, described above, was hastily assembled. There are many ways it should be improved.

1. **Document collection:** Students initially reviewed their designs using a RAG that queried 10,000 pages of related documents that we provided to them. We feel students would have learned more if they had collected the corpus themselves.
2. **Continuous active enhancement by developers:** More importantly, for the future use of COMPASS, we scored the student projects after they submitted artifacts. This was unnecessary, and it missed an important feedback loop. If developers can access to the COMPASS scoring prompts before delivery, they could use active enhancement to improve their systems on a day-to-day basis.
3. **Continuous portfolio-level analytics by management:** Further, if educators (or top level industrial project managers) had access to the artifacts as they are being built, then those educators/managers would apply portfolio-level analytics to offer useful suggestions on how a whole class/industrial group could improve their software.

We use the term **Continuous Intent Verification (CIV)** to describe the above steps 2 and 3.